SPRING FRAMEWORK

- **Spring is a lightweight, open-source, Java-based framework that simplifies enterprise application development.**

- **It was developed by Rod Johnson in 2003.**

- **It is mainly used to develop enterprise-level applications.**

- **It is called a "framework of frameworks" because it provides integration and support for various other frameworks such as Hibernate, JPA etc.**

- **In simple terms, the Spring framework can be defined as a comprehensive structure that provides solutions to various technical problems in application development.**

- **The Spring framework consists of several core modules, including: IOC (Inversion of Control) , Web MVC , AOP , DAO etc.**

Advantages of Spring Framework

- **Predefined Templates : Spring provides ready-to-use templates for technologies such as JDBC, Hibernate, JPA, etc. This reduces boilerplate code by hiding common steps (like opening/closing connections).**
- **Loose Coupling : Spring applications are loosely coupled due to Dependency Injection (DI). This improves flexibility and makes code easier to maintain.**
- **Easy Testing : Thanks to Dependency Injection, testing Spring applications becomes simpler.**
- **Lightweight : Spring is lightweight as it is based on POJOs (Plain Old Java Objects). It doesn't force developers to extend specific classes or implement heavy interfaces.**
- **Integration with Other Frameworks : Spring provides seamless support for integrating with other frameworks and technologies like Hibernate, JPA etc. This makes Java EE application development faster and easier.**

Inversion Of Control (IOC)

Design Patterns in Spring:

- Spring uses design pattern such as IOC , Dependency Injection (DI) to remove dependency from programming code.
- This makes applications easier to test and maintain.

IoC Container :

- It is a spring container that creates the objects , configures, assembles their dependencies and manages their entire life cycle.
- The IoC container is the core of the Spring framework.
- It uses Dependency Injection (DI) to manage application components.

** **NOTE** : The control of object creation and dependency management is given to the Spring container instead of the programmer, hence it is known as Inversion Of Control.

Responsibilities of IoC Container:

Creating objects.
Configuring objects and assembling their dependencies.
Managing the entire lifecycle of objects.

** **NOTE** : The objects created and managed by the IoC container are called Spring Beans.

Types of IoC Containers

Spring provides two main types of IoC containers:

1. **BeanFactory (Core Container)**
   - The most basic IoC container.
   - Provides configuration to instantiate, configure, and manage beans.
   - Loads bean definitions only from XML configuration.
   - Does not support annotation-based configuration.

2. **ApplicationContext (Advanced Container)**
   - Sub-interface of BeanFactory (thus supports all BeanFactory features).
   - Provides additional enterprise-specific features.
   - Supports both XML and annotation-based configurations.
   - Preferred in real-time applications because it is more powerful and feature-rich.

- **Ways to Provide Object Creation Information to IoC Container**
  - **Using XML configuration file**
  - **Using Annotations**

Steps to create a Spring core project : Using XML file

1. **Create a simple maven project and add spring context dependency.**

```
<!--https://mvnrepository.com/artifact/org.springframework/spring-context -->
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.1.14</version>
</dependency>
```

Example 1 :

- **Create a POJO class (class for which you want spring IOC to create object).**

```
public class Sample {

    public void test() {
        System.out.println("Non-static test() method of Sample class is executing.");
    }
}
```

- **Create an xml file and configure the bean.**

(xml file should be created directly inside src/main/java folder only.)

(xml file can have any name).

Config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.springframework.org/schema/context
            http://www.springframework.org/schema/context/spring-context-3.0.xsd">

    <bean id="mysample" class="com.jsp.Sample"></bean>



</beans>
```

- **Create the application class to access the bean by creating IOC container (ApplicationContext) using its implementing classes such as ClassPathXmlApplicatonContext.**

```
public class SampleMain {

    public static void main(String[] args) {

        // create IOC container using ApplicationContext
            // to IOC container , provide xml file as input
        ApplicationContext ac = new ClassPathXmlApplicationContext("Config.xml");

        // from IOC container - ac , getting sample class object
        Sample s = (Sample)ac.getBean("mysample");
                    // OR
        //Sample s = ac.getBean("mysample", Sample.class);

        s.test();
    }
}
```

The xml file name in which information about bean creation is defined.

2. Create a POJO class (class for which you want spring IOC to create object).

```java
package ioc_demo_programs;

public class Student {

    private int id;
    private String name;
    private int age;
    private String address;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }
}
```

3. **Create an xml file and configure the bean.**

(xml file should be created directly inside src/main/java folder only.)

(xml file can have any name).

configure.xml

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation) | htt
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
            http://www.springframework.org/schema/beans/spring-beans.xsd
            http://www.springframework.org/schema/context
            http://www.springframework.org/schema/context/spring-context-3.0.xsd">

    <bean id="mystudent" class="ioc_demo_programs.Student"></bean>



</beans>
```

- **Create the application class to access the bean by creating IOC container (ApplicationContext) using its implementing classes such as ClassPathXmlApplicatonContext.**

```
public class Main {

    public static void main(String[] args) {

        // creating IOC container
        ApplicationContext ac = new ClassPathXmlApplicationContext("configure.xml");

        // accessing/ fetching the bean(object) from IOC container.
        Student s = (Student)ac.getBean("mystudent");

        System.out.println("Student Name : "+s.getName());
        System.out.println("Student age : "+s.getAge());
        System.out.println("Student address : "+s.getAddress());

    }
}
```

- **Dependency Injection(DI) : is the main functionality provided by IOC container.**

- **It is a design pattern that allows the spring container to inject objects into other objects during run time.**

- **The control of creating objects and managing the components is taken care by IOC container with the help of Dependency Injection.**

- **The DI emphasizes on keeping the java classes independent of each other and the container frees them from object creation and maintenance.**

- **Hence , ensuring loose coupling between classes and objects.**

Need for dependency injection ?

Suppose class One needs the object of class Two to instantiate or operate a method, then class One is said to be dependent on class Two.

Now though it might appear okay to depend on a module on the other, in the real world, this could lead to a lot of problems, including system failure. Hence such dependencies need to be avoided.

Spring IOC resolves such dependencies with Dependency Injection, which makes the code easier to test and reuse .

- **Example: Teacher and Marker**
- **Without Dependency Injection (Normal Java) –**

Here, the Teacher depends on Marker and creates it herself. If the marker changes to a whiteboard marker, you must edit the Teacher class.

```
class Teacher {
    Marker marker = new Marker();  // Teacher creates her own marker

    void teach() {
        marker.write();
    }
}
```

- **With DI (Spring Way) –**

```
class Teacher {
    Marker marker;  // Teacher needs a marker, but doesn't create it

    // Marker is given (injected) from outside
    public Teacher(Marker marker) {
        this.marker = marker;
    }

    void teach() {
        marker.write();
    }
}
```

Now, Spring will create the Marker object and give it to the Teacher. Teacher just uses it — doesn't worry about how it's made. That's Dependency Injection — dependencies (Marker) are injected from outside(from IOC).